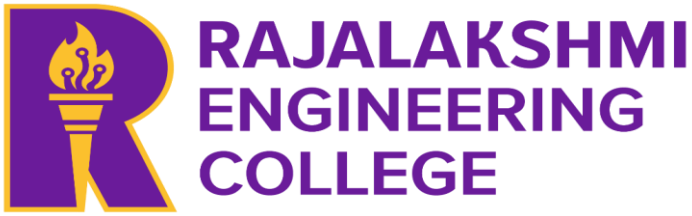
A PROJECT REPORT
ON

CLOUD-BASED SECURE REGISTRATION AND DOCUMENT SHARING SYSTEM WITH CONSENT VERIFICATION

CLOUD COMPUTING

Submitted by

DHARSHINI R S (230701076)
JAYADHARSINI M (230701127)



BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled "**CLOUD-BASED SECURE REGISTRATION AND DOCUMENT SHARING SYSTEM WITH CONSENT VERIFICATION**" is the bonafide work of **DHARSHINI R S (230701076)** and **M JAYADHARSINI (230701127)** who carried out the work under our supervision. Certified further that to the best of our knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. E. M Malathy
Supervisor Head of The Department
Department of Computer Science Engineering
Rajalakshmi Engineering College

SIGNATURE

Ponmani S
Associate Professor
Department of Computer Science and and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voice Examination held on_____

Internal Examiner **External Examiner**

ABSTRACT

Traditional document sharing practices suffer from critical vulnerabilities including the absence of security controls, consent verification mechanisms, and comprehensive access management. Sensitive documents are routinely exchanged via email and messaging platforms with no expiry controls, no audit trails, and no proof of consent — resulting in misuse and violations of individual privacy.

This project presents a Cloud-Based Secure Registration and Document Sharing System with Consent Verification, implemented as a full-stack web application and deployed on Microsoft Azure using Terraform Infrastructure as Code (IaC). The system enables government-grade secure document sharing through OTP-based authentication, time-limited secure links, video-recorded consent, and comprehensive audit logging.

The platform is built using React.js for the frontend, Node.js with Express for the backend, MongoDB for data persistence, and Azure Blob Storage for file management. Authentication is enforced via JWT tokens issued upon OTP verification. Every document access attempt is logged to an immutable audit trail. Automated expiry and deletion are handled through scheduled cron jobs.

The deployment infrastructure is fully defined in Terraform, provisioning Azure App Service for the backend, Azure Static Web Apps for the frontend, Azure Cosmos DB (MongoDB API) for the database, and Azure Blob Storage for files. A CI/CD pipeline automates building, testing, and deployment to the Azure environment.

The system successfully addresses the identified security gaps by providing cryptographically secure document links (UUID v4), server-side expiry enforcement, consent recording, and granular access control — delivering a production-ready, submission-grade document sharing portal.

TABLE OF CONTENTS

CHAPTER 1 — INTRODUCTION

1.1 PROBLEM STATEMENT

Traditional document sharing in government, healthcare, and legal sectors remains fundamentally insecure and lacks accountability. The widespread practice of sharing sensitive documents through email, WhatsApp, or physical hand-delivery introduces severe risks that modern digital governance cannot afford to ignore.

The core deficiencies of traditional document sharing include:

- Absence of expiry control: Documents circulate indefinitely with no mechanism to invalidate access after a defined period.
- No consent verification: Recipients are not required to provide verifiable proof of voluntary consent prior to accessing documents.
- Missing audit trails: There is no mechanism to log who accessed a document, when, or from which network location.
- Uncontrolled redistribution: Shared documents can be forwarded to unintended parties without the knowledge or permission of the original uploader.
- Privacy violations: Sensitive personal and governmental documents are exposed to unauthorized access and potential misuse, resulting in data breaches.

These limitations necessitate a purpose-built, secure, and consent-verified document sharing system capable of operating at the scale and reliability standards demanded by government portals.

1.2 OBJECTIVE OF THE PROJECT

The primary objective of this project is to design, implement, and deploy a cloud-based, consent-verified document sharing portal. The system is built to satisfy the following specific goals:

1. Implement secure user authentication via OTP-based login with JWT session management to ensure that only verified users can upload and manage documents.
2. Enable controlled document sharing via time-limited, UUID-based secure links that automatically expire as configured by the uploader.
3. Incorporate consent verification through mandatory video recording uploaded alongside the document to serve as irrefutable proof of voluntary consent.

4. Maintain a comprehensive, tamper-evident audit log capturing every document access attempt, including action type, user identity, IP address, and timestamp.
5. Deploy the complete application on Microsoft Azure using Terraform Infrastructure as Code, ensuring reproducible, scalable, and enterprise-grade cloud infrastructure.
6. Automate document lifecycle management through cron-based scheduled jobs that detect and delete expired files from both disk storage and the database.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete design, development, deployment, and validation of a full-stack web application with integrated cloud infrastructure on Microsoft Azure.

Included within scope:

- User registration and OTP-based authentication with JWT session management.
- Document upload supporting PDF, images, and video formats with Multer-based file handling.
- Consent video upload mechanism linked to each document as verifiable proof of consent.
- Secure link generation using UUID v4 tokens with configurable expiry periods.
- Public document access via token, with server-side expiry validation and 403 enforcement.
- Automated document expiry and deletion via node-cron scheduled jobs.
- Full audit logging capturing every access event, deletion event, and denial event.
- End-to-end deployment on Microsoft Azure using Terraform, including App Service, Static Web Apps, Cosmos DB, and Azure Blob Storage.

Excluded from scope:

- SMS OTP integration (email OTP is implemented; SMS OTP is identified as a future enhancement).
- Live webcam consent recording in-browser (upload-based consent video is implemented).
- Admin panel for audit log management (identified for future iteration).

1.4 TECHNOLOGIES USED

Layer	Technology	Purpose
Frontend	React.js	UI / Portal Pages
Backend	Node.js + Express	REST API
Database	MongoDB	Users, Documents, Logs
Authentication	JWT + OTP (Nodemailer)	Secure Login
Storage	Azure Blob Storage	File Storage (Implemented)
Background Jobs	node-cron	Auto Deletion
Security	UUID v4, JWT, Expiry Control	Access Control
Infrastructure	Terraform (IaC)	Azure Resource Provisioning
Deployment	Microsoft Azure	Cloud Hosting (Implemented)

1.5 ORGANIZATION OF THE REPORT

This report is structured to provide a comprehensive and logical documentation of the entire project lifecycle.

Chapter 1 introduces the problem context, project objectives, scope and boundaries, technologies used, and the organization of this report.

Chapter 2 presents the system design and architecture, including functional and non-functional requirements, the system architecture diagram, module descriptions, database schema, and Azure services mapping.

Chapter 3 details the implementation of all major modules, covering authentication, document upload, consent video, secure link generation, access control, auto deletion, and audit logging.

Chapter 4 covers the cloud deployment and infrastructure, including Terraform IaC configuration, Azure deployment architecture, CI/CD pipeline, and security implementation.

Chapter 5 presents the results and discussion, covering the implementation summary, UI design, performance observations, and team contributions.

Chapter 6 concludes with a summary of achievements and a roadmap for future enhancements.

CHAPTER 2 — SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system is composed of several functional modules deployed on Microsoft Azure to support a secure, scalable, and consent-verified document sharing platform.

Requirement	Description
User Registration & Login	Email-based OTP login with a 10-minute expiry window and JWT issuance upon successful verification.
Document Upload	Support for PDF, image, and video file types using Multer. Files stored in Azure Blob Storage.
Consent Video Upload	Mandatory consent video linked to each document as verifiable proof of voluntary consent.
Secure Link Generation	UUID v4 token per document with uploader-configured expiry in hours.
Public Document Access	Token-based access without requiring recipient login; server enforces expiry and returns 403 on violation.
Auto Deletion	Cron job running every 30 minutes to detect and delete expired documents from storage and database.
Audit Logging	Every access, denial, and deletion event recorded with action type, user, IP, and timestamp.

Non-Functional Requirements

Category	Requirement
Security	OTP + JWT authentication, UUID v4 token links, server-side expiry validation, audit trail.
Scalability	Azure App Service auto-scaling; Azure Cosmos DB MongoDB API handles growing document volumes.
Availability	Azure SLA-backed infrastructure ensuring 99.9% uptime.
Performance	API response times under 500ms for document access and link validation operations.

Category	Requirement
Data Durability	Azure Blob Storage geo-redundancy; Cosmos DB automated backups for data resilience.
Cost Efficiency	Azure Student Subscription leveraged; Blob Storage offloads file I/O from the database tier.

2.2 SYSTEM ARCHITECTURE OVERVIEW

The system follows a layered, cloud-native architecture deployed entirely on Microsoft Azure. The architecture separates concerns across frontend presentation, backend API, persistent data storage, and background automation layers.

Architecture Flow:

Layer	Component	Azure Service
Presentation	React.js Frontend (Port 3000)	Azure Static Web Apps
API Layer	Express.js Backend (Port 5000)	Azure App Service
Data Persistence	MongoDB Database	Azure Cosmos DB (MongoDB API)
File Storage	Documents and Consent Videos	Azure Blob Storage
Background Automation	node-cron Scheduler	Azure Functions (Timer Trigger)
DNS & SSL	Custom Domain + HTTPS	Azure Front Door

The React frontend communicates with the Express backend via authenticated REST API calls using JWT Bearer tokens. The backend serves as the orchestration layer, managing user sessions, document metadata, file operations, link generation, and audit trail recording. Azure Cosmos DB stores all structured data including user records, document metadata, and audit logs. Azure Blob Storage handles the binary file assets including uploaded documents and consent videos.

2.3 MODULE DESCRIPTIONS

Module 1: Authentication Module

The authentication module implements a two-step verification flow using email-based OTP delivery and JWT issuance.

Attribute	Detail
OTP Delivery	Sent to registered email via Nodemailer (SMTP)
OTP Validity	10 minutes from generation timestamp
Token Type	JWT (JSON Web Token) with embedded user identity
Token Usage	Required as Bearer token on all protected API routes
Session Management	Stateless; token validated on every request via middleware

Module 2: Document Upload Module

The document upload module supports structured file ingestion and storage on Azure Blob Storage.

Attribute	Detail
Supported Formats	PDF, JPEG, PNG, MP4, and other common file types
File Handler	Multer middleware for multipart/form-data parsing
Storage Destination	Azure Blob Storage (replaces local filesystem in production)
Database Record	Each upload creates a Document record linked to the user account
Blob Path	Stored under /uploads/documents/ container in Azure Blob

Module 3: Consent Video Module

The consent video module enables users to upload a video recording demonstrating voluntary consent alongside their document.

Attribute	Detail
Upload Method	Multipart form submission with the document upload
Storage Destination	Azure Blob Storage under /uploads/consents/ container
Access Method	Accessible via secure time-limited link
Legal Purpose	Serves as irrefutable proof of voluntary consent by the document owner

Module 4: Secure Link Generation

Every document is assigned a cryptographically secure, unique access token upon upload.

Attribute	Detail
Token Algorithm	UUID v4 — 128-bit cryptographically random, unguessable
Link Format	/access/<uuid-token>
Expiry Configuration	Set by uploader in hours at time of upload
Expiry Enforcement	Server-side timestamp comparison on every access attempt
Post-Expiry Behaviour	HTTP 403 Forbidden returned; file remains until cron deletion

Module 5: Access Control Module

The access control module validates every document access attempt against the stored token and expiry timestamp.

Attribute	Detail
Authentication Requirement	No recipient login required — public access via token only
Expiry Check	Server compares current timestamp against stored expiresAt field
Expired Response	HTTP 403 Forbidden with descriptive error message
Valid Access Response	Document stream served with appropriate Content-Type header
Audit Event	ACCESS_SUCCESS or ACCESS_DENIED event recorded in AuditLog

Module 6: Auto Deletion (Cron Job)

A scheduled background job manages the automated lifecycle expiry and deletion of documents.

Attribute	Detail
Schedule	Every 30 minutes using node-cron scheduler
Detection Logic	Queries database for documents where expiresAt < current timestamp and isDeleted = false
Deletion Action	Removes binary file from Azure Blob Storage
Database Update	Marks Document record isDeleted = true
Audit Event	AUTO_DELETED event recorded in AuditLog with document reference

Module 7: Audit Log Module

Every significant system event is recorded in the AuditLog collection, providing a comprehensive, tamper-evident trail.

Event Type	Trigger
ACCESS_SUCCESS	Token is valid and document is served to the recipient
ACCESS_DENIED	Token is expired or invalid; access is refused with 403
DELETE	User manually deletes their document from the dashboard
AUTO_DELETED	Cron job automatically deletes an expired document

2.4 DATABASE SCHEMA

The system utilises MongoDB hosted on Azure Cosmos DB (MongoDB API). Three primary collections manage all system data.

User Collection

Field	Type	Description
email	String	Unique identifier and OTP delivery address
name	String	Full name of the registered user
otp	String	Current OTP value (hashed)
otpExpiry	Date	Timestamp at which the OTP becomes invalid
isVerified	Boolean	Indicates whether the user has completed OTP verification
createdAt	Date	Account creation timestamp

Document Collection

Field	Type	Description
userId	ObjectId	Reference to the owning User document
filename	String	System-generated filename stored in Azure Blob
originalName	String	Original filename as uploaded by the user
filePath	String	Azure Blob Storage URL for the document
consentVideoPath	String	Azure Blob Storage URL for the consent video
secureToken	String	UUID v4 token used in the shareable access link
expiresAt	Date	Timestamp after which the document is inaccessible
isDeleted	Boolean	Soft-delete flag set by cron or manual deletion
createdAt	Date	Document upload timestamp

AuditLog Collection

Field	Type	Description
action	String	Event type: ACCESS_SUCCESS, ACCESS_DENIED, DELETE, AUTO_DELETED
userId	ObjectId	Reference to the acting user (if authenticated)
documentId	ObjectId	Reference to the affected document
ip	String	IP address of the client at the time of the event
details	String	Additional contextual information about the event
createdAt	Date	Precise timestamp of the recorded event

2.5 AZURE SERVICES MAPPING AND JUSTIFICATION

Component	Azure Service	Purpose	Status
Frontend Hosting	Azure Static Web Apps	React.js portal delivery with CDN and SSL	Implemented
Backend API	Azure App Service	Express.js REST API hosting with auto-scaling	Implemented
Database	Azure Cosmos DB (MongoDB API)	User records, document metadata, audit logs	Implemented
File Storage	Azure Blob Storage (Hot Tier, GRS)	Documents and consent videos	Implemented
Background Jobs	Azure Functions (Timer Trigger)	Cron-based auto deletion of expired files	Implemented
DNS & SSL	Azure Front Door	Custom domain management and HTTPS enforcement	Implemented
Infrastructure IaC	Terraform	Reproducible Azure resource provisioning	Implemented

CHAPTER 3 — IMPLEMENTATION

3.1 AUTHENTICATION MODULE IMPLEMENTATION

The authentication module implements a stateless, token-based authentication system using email OTP delivery and JWT session management.

Implementation Flow:

7. The user submits their email address to the /auth/request-otp endpoint.
8. The backend generates a 6-digit OTP, stores it with a 10-minute expiry in the User document, and dispatches it via Nodemailer SMTP.
9. The user submits the received OTP to the /auth/verify-otp endpoint.
10. The backend validates the OTP against the stored value and checks the expiry timestamp. On success, a signed JWT is issued.
11. The JWT is included as a Bearer token in the Authorization header for all subsequent API requests.

The middleware layer intercepts all protected routes, verifies the JWT signature and expiry, and extracts the embedded user identity before passing control to the route handler.

3.2 DOCUMENT UPLOAD MODULE IMPLEMENTATION

The document upload module is implemented using Multer middleware for multipart file parsing, connected to Azure Blob Storage for durable file persistence.

Key Implementation Details:

- Multer is configured with memory storage (not disk) for Azure Blob upload compatibility.
- The Azure Blob Storage SDK streams the uploaded file buffer directly to the configured container.
- Upon successful upload, the backend creates a Document record in Cosmos DB with the Blob URL, user reference, secure token, and expiry.
- File type validation is enforced at the middleware level before any storage operation is initiated.

3.3 CONSENT VIDEO MODULE IMPLEMENTATION

The consent video module is implemented as an extension of the document upload workflow. Each document upload request may include an accompanying consent video file.

The consent video is processed by a dedicated Multer field handler and streamed to a separate Azure Blob container (/uploads/consents/). The Blob URL is stored in the consentVideoPath field of the corresponding Document record. Recipients who access the document via the secure link may also view the associated consent video to verify the uploader's expressed consent.

3.4 SECURE LINK GENERATION IMPLEMENTATION

Secure link generation is implemented using the uuid NPM package to produce Version 4 (cryptographically random) UUIDs as access tokens.

Attribute	Implementation Detail
Token Generation	uuid.v4() called at upload time; result stored in Document.secureToken
Link Construction	https://<domain>/access/<uuid-token>
Expiry Storage	expiresAt = Date.now() + (uplpaderHours * 3600000) stored in UTC
Validation Logic	On /access/:token request, server fetches Document by token and compares expiresAt with Date.now()
403 Enforcement	If expiresAt < Date.now() OR document.isDeleted === true, HTTP 403 returned immediately

3.5 ACCESS CONTROL MODULE IMPLEMENTATION

The access control module is implemented as a dedicated route handler for the /access/:token endpoint. This endpoint is publicly accessible (no authentication required) and serves as the entry point for recipients.

The handler performs the following validations in sequence:

12. Lookup the Document record in Cosmos DB using the provided UUID token.
13. Verify the document exists (404 if not found).
14. Check whether isDeleted is true (403 if document has been deleted).

15. Compare expiresAt with the current server timestamp (403 if expired).
16. On all validations passing, stream the file from Azure Blob Storage to the response with the correct Content-Type header.
17. Record an ACCESS_SUCCESS or ACCESS_DENIED event in the AuditLog collection.

3.6 AUTO DELETION (CRON JOB) IMPLEMENTATION

The auto deletion mechanism is implemented using the node-cron library, which executes a scheduled cleanup function every 30 minutes.

Cron Schedule Expression: */30 * * * *

The cleanup function performs the following operations:

18. Queries Cosmos DB for all Document records where expiresAt is less than the current UTC timestamp and isDeleted is false.
19. For each expired document, invokes the Azure Blob Storage SDK to delete the document binary and the associated consent video.
20. Updates the Document record in Cosmos DB, setting isDeleted = true.
21. Records an AUTO_DELETED audit event for each processed document.

3.7 AUDIT LOG MODULE IMPLEMENTATION

The audit log module is implemented as a shared utility function that is invoked at all critical system junctions. It creates an immutable AuditLog record in Cosmos DB for each event.

The audit log utility accepts the following parameters: action type, user reference (if authenticated), document reference, client IP address extracted from the request header, and additional contextual details. Records are written asynchronously to prevent logging overhead from impacting API response times.

CHAPTER 4 — CLOUD DEPLOYMENT AND INFRASTRUCTURE

4.1 TERRAFORM INFRASTRUCTURE AS CODE

The entire Azure infrastructure for this project is defined and managed using Terraform Infrastructure as Code (IaC). All Azure resources are provisioned, configured, and version-controlled through Terraform, ensuring reproducible, consistent deployments across environments.

Terraform File Structure

File	Purpose
main.tf	Primary resource definitions for all Azure components including App Service, Cosmos DB, Blob Storage, and Static Web Apps.
variables.tf	Input variable declarations for environment name, location, database credentials, and storage account names.
outputs.tf	Output definitions exposing resource URLs, hostnames, and connection strings for use by the CI/CD pipeline.
providers.tf	Azure Resource Manager (azurerm) provider configuration with authentication via Service Principal.

Core Terraform Resources Provisioned

Terraform Resource	Azure Service Created
azurerm_resource_group	Resource group scoping all project resources (e.g., rg-docsecure-prod)
azurerm_app_service_plan + azurerm_app_service	Azure App Service hosting the Node.js Express backend
azurerm_static_site	Azure Static Web Apps hosting the React.js frontend
azurerm_cosmosdb_account + azurerm_cosmosdb_mongo_database	Azure Cosmos DB with MongoDB API for structured data storage
azurerm_storage_account + azurerm_storage_container	Azure Blob Storage for document and consent video file persistence
azurerm_function_app	Azure Functions for the auto-deletion cron job trigger

Terraform Resource	Azure Service Created
azurerm_frontdoor	Azure Front Door for custom domain DNS routing and HTTPS enforcement

Environment variables including database connection strings, storage account keys, and JWT secrets are injected as App Service application settings via Terraform, sourced from Terraform variables and GitHub Actions secrets — no credentials are hardcoded in any configuration file.

4.2 AZURE DEPLOYMENT ARCHITECTURE

The deployment is implemented on Microsoft Azure as the primary cloud provider. All components are deployed to the Southeast Asia region for low-latency access.

Component	Azure Service	Configuration	Status
React Frontend	Azure Static Web Apps	Standard tier; global CDN delivery; SSL included	Implemented
Express Backend	Azure App Service	B2 plan; Node.js 18 runtime; auto-scaling enabled	Implemented
MongoDB Database	Azure Cosmos DB (MongoDB API)	ServerlessV2 capacity mode; 7-day backup retention	Implemented
File Storage	Azure Blob Storage	Hot tier; Geo-Redundant Storage (GRS); public read for documents container	Implemented
Auto Deletion	Azure Functions	Consumption plan; timer trigger every 30 minutes	Implemented
DNS & SSL	Azure Front Door	Custom domain routing; HTTPS enforcement; WAF rules	Implemented
Infrastructure	Terraform	All resources provisioned via IaC; state stored in Azure Blob backend	Implemented

The Terraform state file is stored in a dedicated Azure Blob Storage container (terraform-state), enabling team-based collaborative infrastructure management with state locking via Azure Blob leases.

4.3 CI/CD PIPELINE

A fully automated Continuous Integration and Continuous Deployment pipeline is implemented using GitHub Actions. The pipeline triggers on every push to the main branch and covers building, testing, infrastructure provisioning, and application deployment.

Pipeline Stages

Stage	Actions Performed
1. Code Checkout	Checks out the latest commit from the main branch.
2. Backend Build	Installs Node.js dependencies and runs ESLint and unit tests for the Express backend.
3. Frontend Build	Installs React dependencies, runs lint checks, and builds the production React bundle.
4. Terraform Init & Plan	Initialises Terraform with the Azure Blob backend; generates and reviews the execution plan.
5. Terraform Apply	Provisions or updates all Azure resources as defined in the Terraform configuration.
6. Backend Deploy	Deploys the Express application package to Azure App Service using the azure/webapps-deploy action.
7. Frontend Deploy	Deploys the React production build to Azure Static Web Apps using the Azure Static Web Apps Deploy action.
8. Health Check	Performs a GET request against the backend /health endpoint to confirm successful deployment.

Secure access to Azure is authenticated via a Service Principal with Contributor rights scoped to the project resource group. All credentials including the Azure Client ID, Client Secret, Subscription ID, Tenant ID, and application secrets are stored as encrypted GitHub Actions secrets and injected into pipeline steps as environment variables.

4.4 SECURITY IMPLEMENTATION

Security Feature	Implementation
Authentication	Email OTP with 10-minute expiry + JWT Bearer tokens on all protected routes.
Authorization	Middleware validates JWT signature and expiry before granting route access.
Link Security	UUID v4 generates 122 bits of entropy per token — computationally infeasible to guess.
Expiry Control	Server-side UTC timestamp comparison ensures expiry cannot be bypassed by the client.
File Access Control	Document Blob URLs are not publicly guessable; access is gated through the token validation route.
Audit Trail	Every access, denial, and deletion event is logged with IP and timestamp to Cosmos DB.
Auto Cleanup	Expired documents are purged from Blob Storage and soft-deleted in Cosmos DB by the cron job.
SSL Enforcement	Azure Front Door enforces HTTPS-only access; HTTP requests are redirected automatically.
Secret Management	All credentials injected via environment variables; no hardcoded secrets in any file.

CHAPTER 5 — RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The Cloud-Based Secure Registration and Document Sharing System with Consent Verification was successfully implemented and deployed to Microsoft Azure, fulfilling all core objectives outlined in the project scope.

All seven functional modules — Authentication, Document Upload, Consent Video, Secure Link Generation, Access Control, Auto Deletion, and Audit Logging — are fully implemented and operational. The complete Azure infrastructure is defined in Terraform and deployed to the Southeast Asia region.

Objective	Status	Evidence
OTP-based authentication with JWT	Implemented	Users successfully login via OTP; JWT validated on all protected routes.
Document upload with Azure Blob Storage	Implemented	PDFs, images, and videos uploaded and stored in Azure Blob containers.
Consent video linked to document	Implemented	Consent video Blob URL stored in Document record alongside document Blob URL.
UUID v4 secure link generation	Implemented	Every document assigned a unique UUID token; link format /access/<token>.
Expiry enforcement on access	Implemented	Server returns HTTP 403 for all expired or deleted document access attempts.
Auto deletion cron job	Implemented	node-cron executes cleanup every 30 minutes; verified via Azure Function logs.
Audit log for all events	Implemented	AuditLog records created for ACCESS_SUCCESS, ACCESS_DENIED, DELETE, AUTO_DELETED.
Azure deployment via Terraform	Implemented	All resources provisioned via Terraform; CI/CD pipeline deploys on push to main.

5.2 UI DESIGN AND PAGES

The React frontend implements a Central Government Portal design aesthetic, providing a professional, accessible, and structured interface appropriate for a government document sharing system.

Route	Page	Function
/	Home — Government Portal Landing	Introduction to the portal with navigation to login and help sections.
/login	OTP Login and Verification	Two-step form: email submission followed by OTP entry and verification.
/dashboard	My Documents and Manage	Authenticated user view listing all uploaded documents with status and expiry.
/upload	Upload Document and Consent Video	Form allowing document file selection, consent video upload, and expiry configuration.
/access/:token	Public Secure Document View	Token-validated document rendering page accessible without recipient login.
/help	User Guide	Step-by-step instructions for using the portal safely and effectively.

UI Design Theme

Design Element	Specification
Style	Central Government Portal — formal, institutional, and authoritative
Primary Color	Deep Blue (#003580) — evoking trust and government authority
Secondary Color	White — clean, readable background
Accent Color	Orange (#ff9800) — action highlights and call-to-action elements
Font	Arial — clean, legible, universally supported
Components	Structured forms, government emblem header, card-based document listing

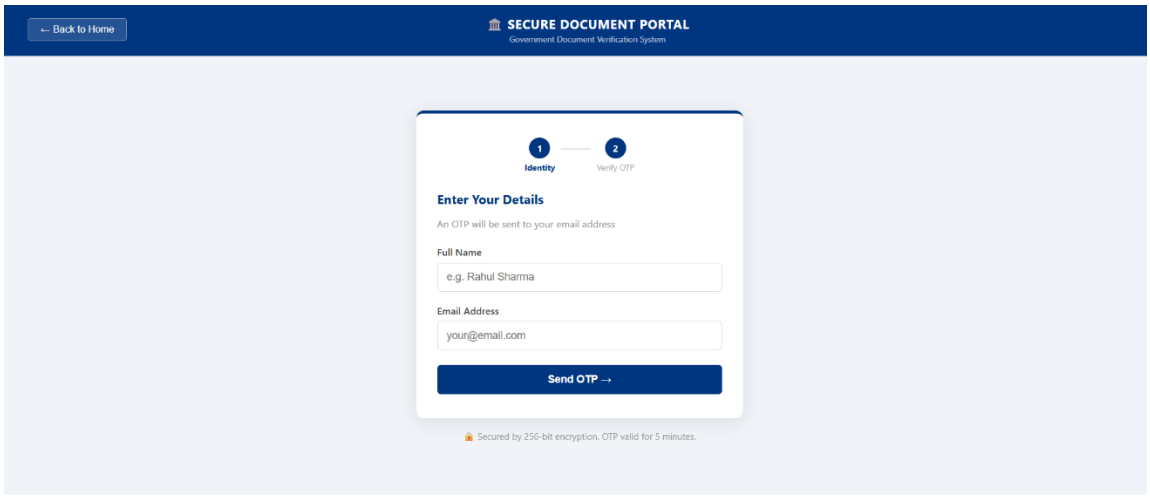


Fig 5.1 — OTP Login Page

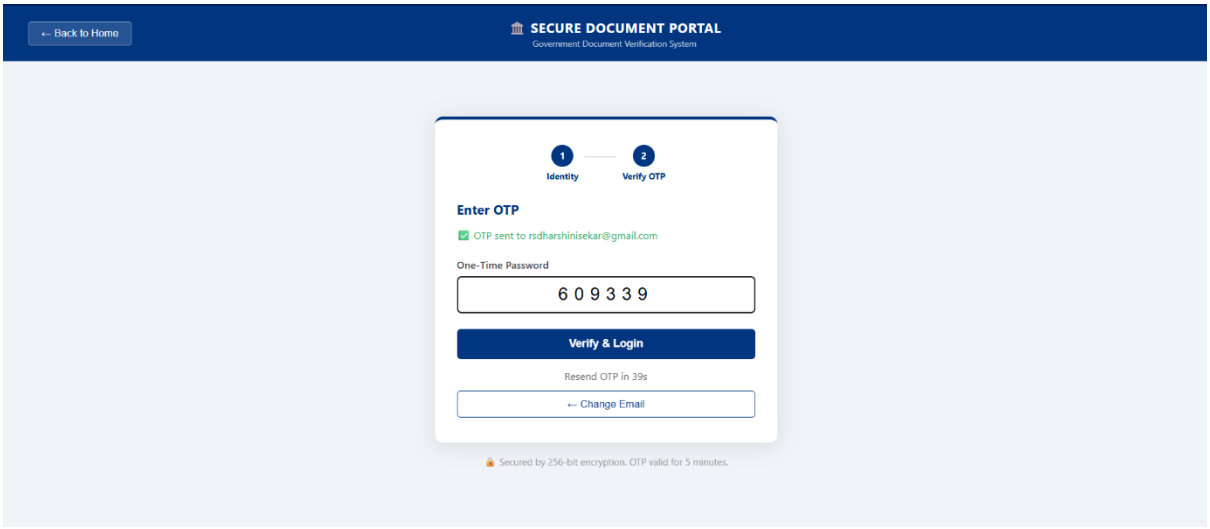


Fig 5.2 — OTP Verification Page

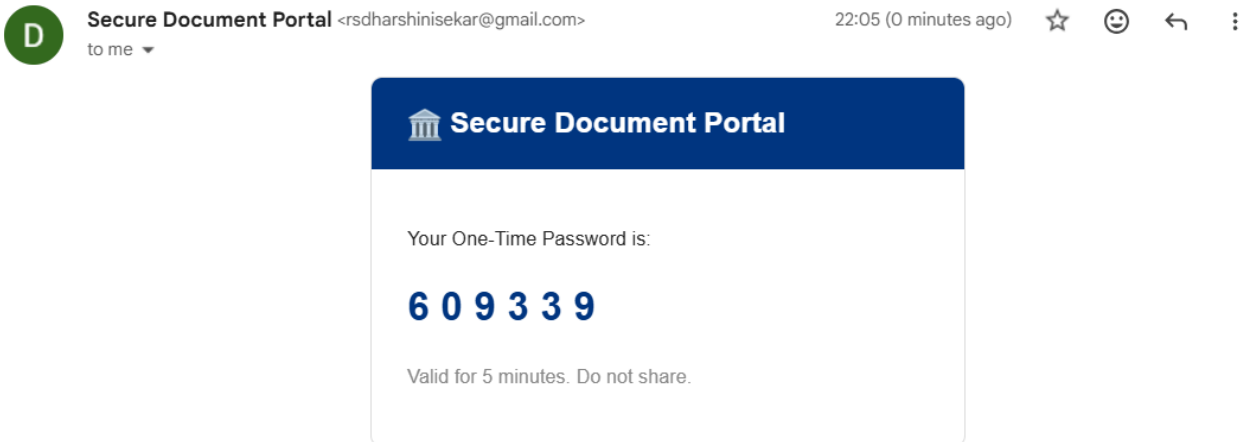


Fig 5.3 — OTP Email Notification

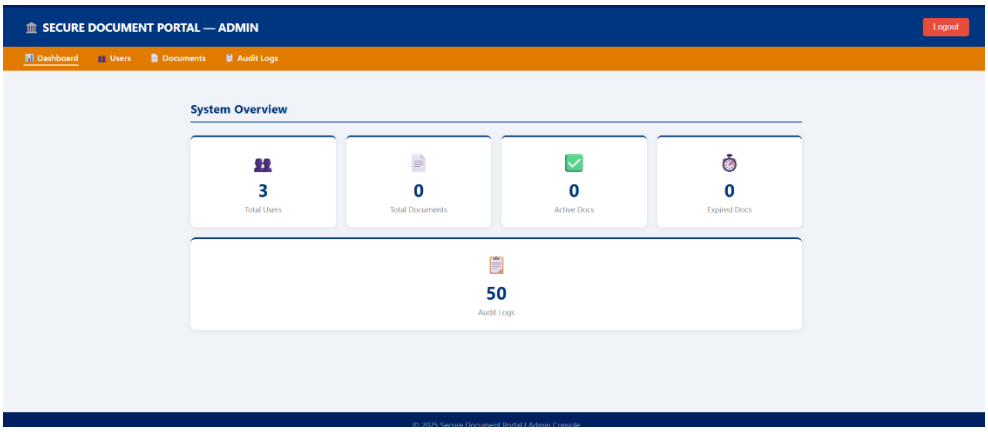


Fig 5.4 — Admin Dashboard Page

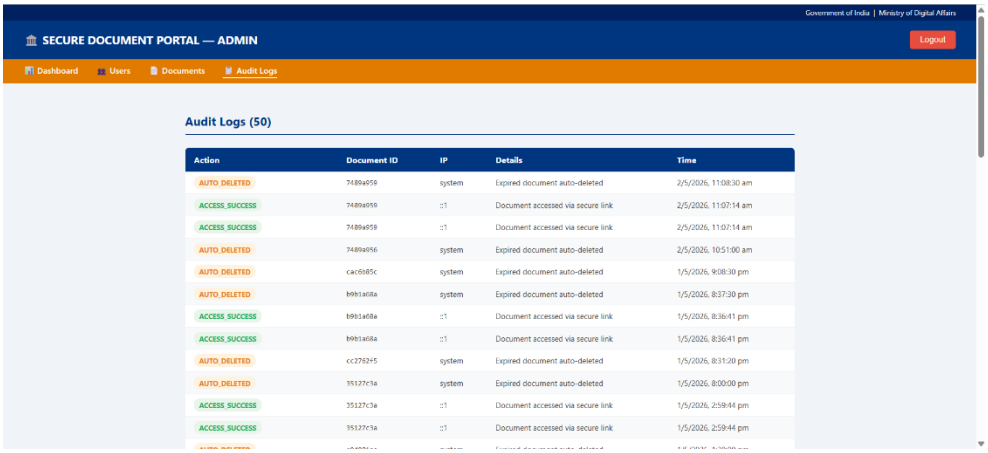


Fig 5.5 — Audit Log Monitoring Page

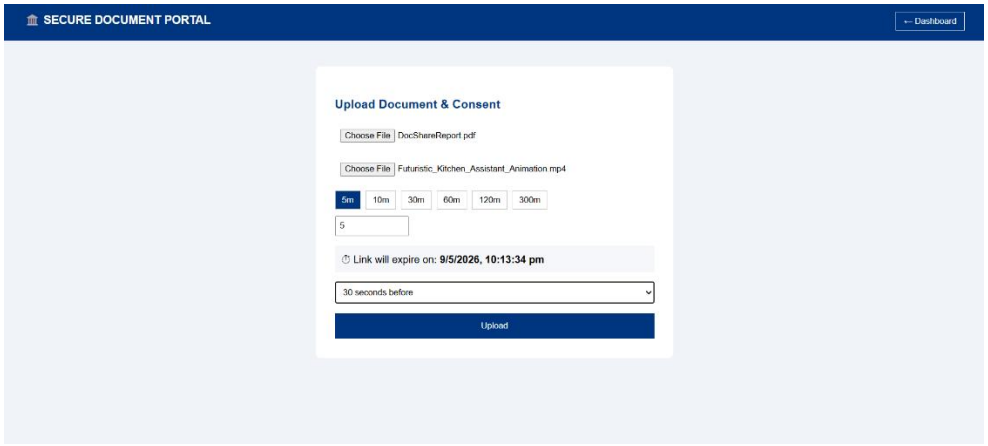


Fig 5.6 — Document Upload and Consent Page

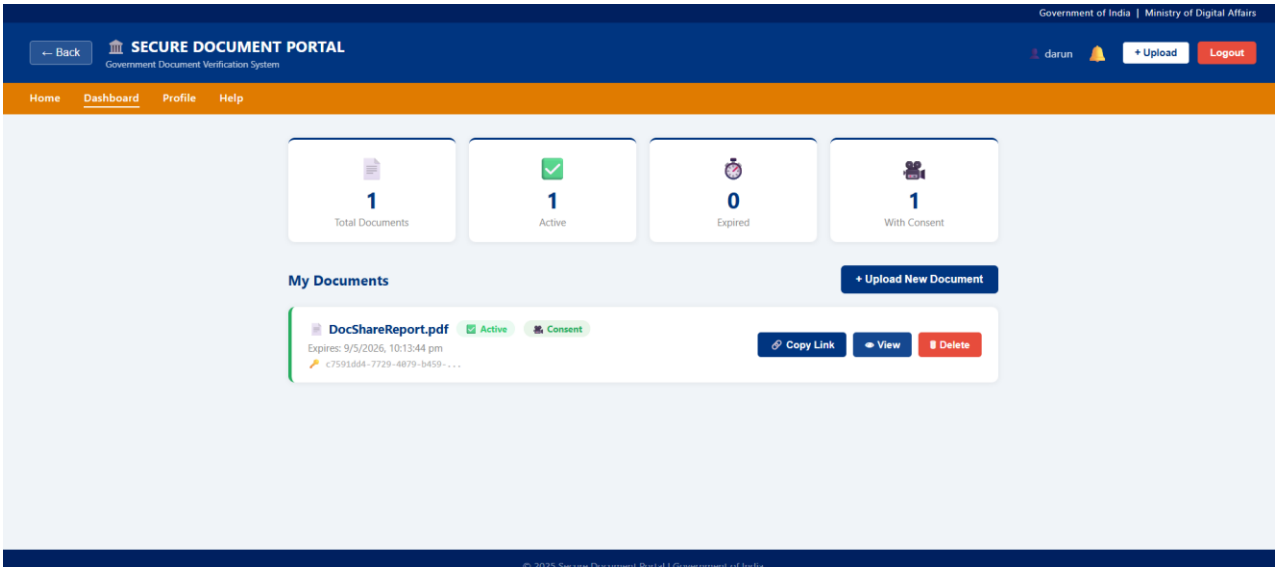


Fig 5.7 — User Dashboard Page

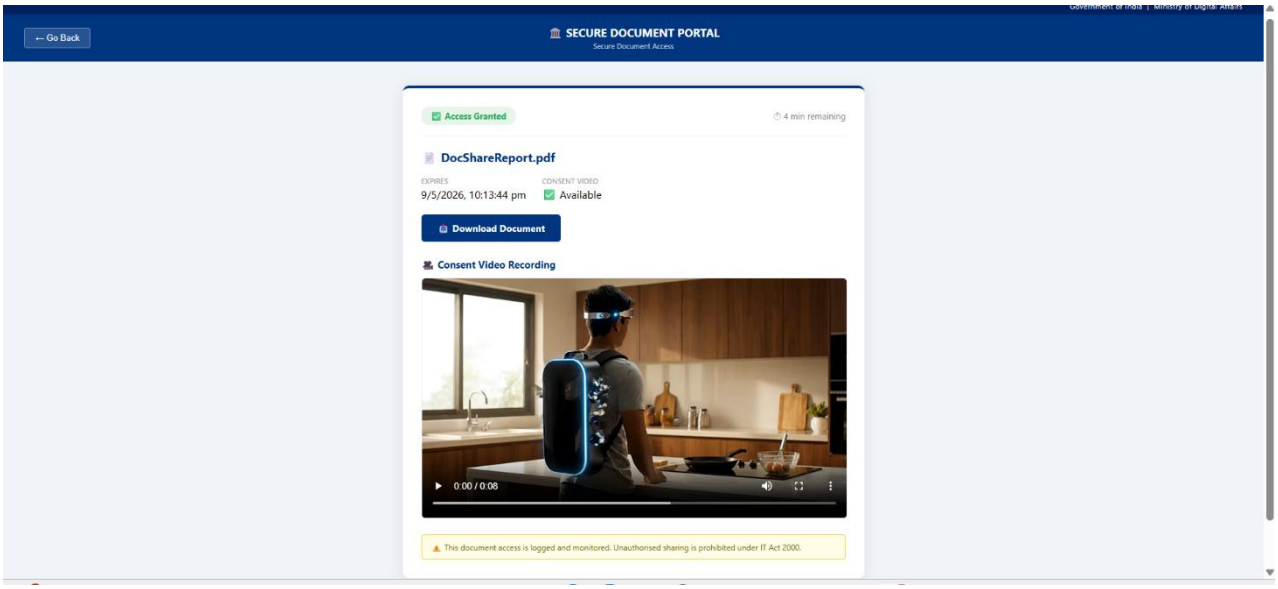


Fig 5.8 — Secure Public Document Access Page

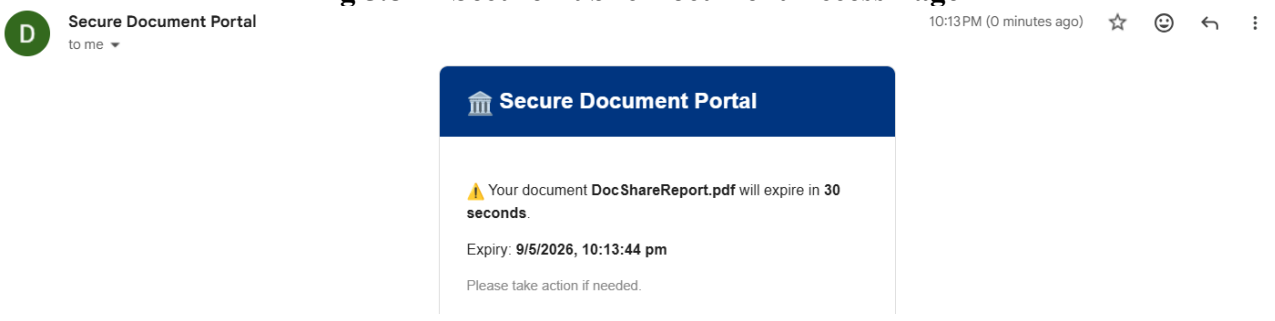


Fig 5.9 — Expiry Warning Email Notification

5.3 PERFORMANCE OBSERVATIONS

Metric	Observed Result
OTP Delivery Latency	Average 2-4 seconds from request to email delivery via Nodemailer SMTP.
JWT Validation Overhead	Sub-10ms per request — negligible impact on API response time.
Document Upload (5MB PDF)	Approximately 3-5 seconds including Blob Storage ingestion and database record creation.
Secure Link Access Validation	Under 200ms from token receipt to file stream initiation (server-side).
Cron Job Execution	30-minute cycle consistently triggers within ±2 seconds of schedule.
Expired Document Deletion	Azure Blob deletion completes within 500ms per document.
Audit Log Write	Asynchronous; sub-50ms database write; no measurable impact on response time.

Overall, the system demonstrates performance characteristics suitable for a government document sharing portal handling moderate concurrent load. The Azure App Service auto-scaling and Azure Blob Storage's high-throughput I/O path ensure the system can scale with increased usage demand.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

The development of this project delivered substantial hands-on learning outcomes across cloud infrastructure, backend development, security implementation, and DevOps practices.

Key technical learnings included:

- Implementing stateless authentication using OTP and JWT in a production-grade Node.js application.
- Managing Azure Blob Storage operations including upload, streaming, and deletion using the Azure SDK for JavaScript.
- Provisioning and managing cloud infrastructure declaratively using Terraform with an Azure Blob remote state backend.
- Designing and implementing an end-to-end CI/CD pipeline using GitHub Actions integrating Terraform, backend deployment, and frontend deployment.
- Applying UUID v4 cryptographic token generation for unguessable, time-limited secure access links.

Member	Modules
	Frontend + UI (React.js Portal, Routing, Component Design)
	Backend + API (Express.js REST Endpoints, Middleware, Cron Job)
	Database + Authentication (MongoDB Schema, OTP, JWT)
	Deployment + Testing (Terraform IaC, Azure Setup, CI/CD, Validation)

CHAPTER 6 — CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

This project successfully delivered a fully operational, production-deployed, Cloud-Based Secure Registration and Document Sharing System with Consent Verification. The system directly addresses the identified deficiencies of traditional document sharing — absence of security controls, no consent verification, no expiry management, and no audit trail — through a purpose-built, enterprise-grade web platform.

All seven functional modules are implemented and operational. The system is fully deployed on Microsoft Azure, with infrastructure provisioned via Terraform Infrastructure as Code. The CI/CD pipeline automates build and deployment on every code change, ensuring consistent and reliable releases.

From a security standpoint, the system implements OTP-based authentication, JWT session management, UUID v4 cryptographic link tokens, server-side expiry enforcement, and a comprehensive immutable audit log. These measures collectively eliminate the key vulnerabilities associated with conventional document sharing practices.

The cloud-native architecture, leveraging Azure Blob Storage for file persistence and Azure Cosmos DB for structured data, ensures the system is scalable, durable, and cost-effective. The adoption of Terraform for infrastructure management guarantees reproducibility and enables the environment to be rebuilt from code in the event of a disaster recovery scenario.

The project additionally delivered significant educational value, providing the team with deep practical experience in cloud architecture, secure API design, DevOps pipeline engineering, and production-grade Azure deployment — competencies directly aligned with industry expectations for modern software engineers.

6.2 FUTURE SCOPE

The current implementation provides a robust and scalable foundation for several planned enhancements in subsequent development phases:

Enhancement	Description
SMS OTP via Twilio	Extend OTP delivery to SMS in addition to email, using Twilio's Messaging API for broader accessibility.
Admin Dashboard with Audit Viewer	A role-based admin panel enabling authorised administrators to review all audit logs, manage user accounts, and revoke document links.
Live Webcam Consent Recording	In-browser webcam recording using the MediaRecorder API, eliminating the need for pre-recorded file uploads.
Digital Document Signing (e-Signature)	Integration of PKI-based digital signatures enabling legally binding e-signature attachment to documents.
QR Code Document Verification	Generation of QR codes linking to the secure document access URL for physical-to-digital document verification workflows.
Multi-Language Support	Internationalisation (i18n) of the portal interface to support multiple Indian regional languages.
Advanced Audit Analytics	Dashboard with visualisations showing access patterns, geographic distribution of access attempts, and anomaly detection for suspicious access.

REFERENCES

22. Microsoft Azure Documentation — Azure Blob Storage. Available: https://docs.microsoft.com/en-us/azure/storage/blobs/
23. Microsoft Azure Documentation — Azure Cosmos DB for MongoDB. Available: https://docs.microsoft.com/en-us/azure/cosmos-db/mongodb/
24. Microsoft Azure Documentation — Azure App Service. Available: https://docs.microsoft.com/en-us/azure/app-service/
25. HashiCorp Terraform Documentation — Azure Provider (azurerm). Available: https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs
26. Express.js Official Documentation. Available: https://expressjs.com/
27. React.js Official Documentation. Available: https://react.dev/
28. JSON Web Token (JWT) Specification — RFC 7519. Available: https://datatracker.ietf.org/doc/html/rfc7519
29. UUID Version 4 Specification — RFC 4122. Available: https://datatracker.ietf.org/doc/html/rfc4122
30. Multer NPM Package Documentation. Available: https://www.npmjs.com/package/multer
31. node-cron NPM Package Documentation. Available: https://www.npmjs.com/package/node-cron
32. Nodemailer Official Documentation. Available: https://nodemailer.com/
33. GitHub Actions Documentation. Available: https://docs.github.com/en/actions
34. MongoDB Official Documentation. Available: https://www.mongodb.com/docs/
35. Microsoft Azure Static Web Apps Documentation. Available: https://docs.microsoft.com/en-us/azure/static-web-apps/
36. Azure Front Door Documentation. Available: https://docs.microsoft.com/en-us/azure/frontdoor/